Programming Fundamentals

Module B - Computation

Le The Anh
anhlt161@fe.edu.vn



FPT UNIVERSITY

Objectives

1. Variables
   - Data Types
   - Representation of Integral Values
   - Declarations
2. Basic Memory Operations
   - Constants
   - Assignment Operators
   - Output
   - Input
3. Expressions
   - Arithmetic Expressions
   - Relational Expressions
   - Logical Expressions
   - Shorthand Assignment Operators
   - Mixed Data Types
   - Casting
   - Precedence

Variables

- A variable is a name reference to a memory location, holds data that can change in value during the lifetime of the variable.
- The C language associates a data type with each variable.
- Each data type occupies a compiler-defined number of bytes.

Primitive Data Types

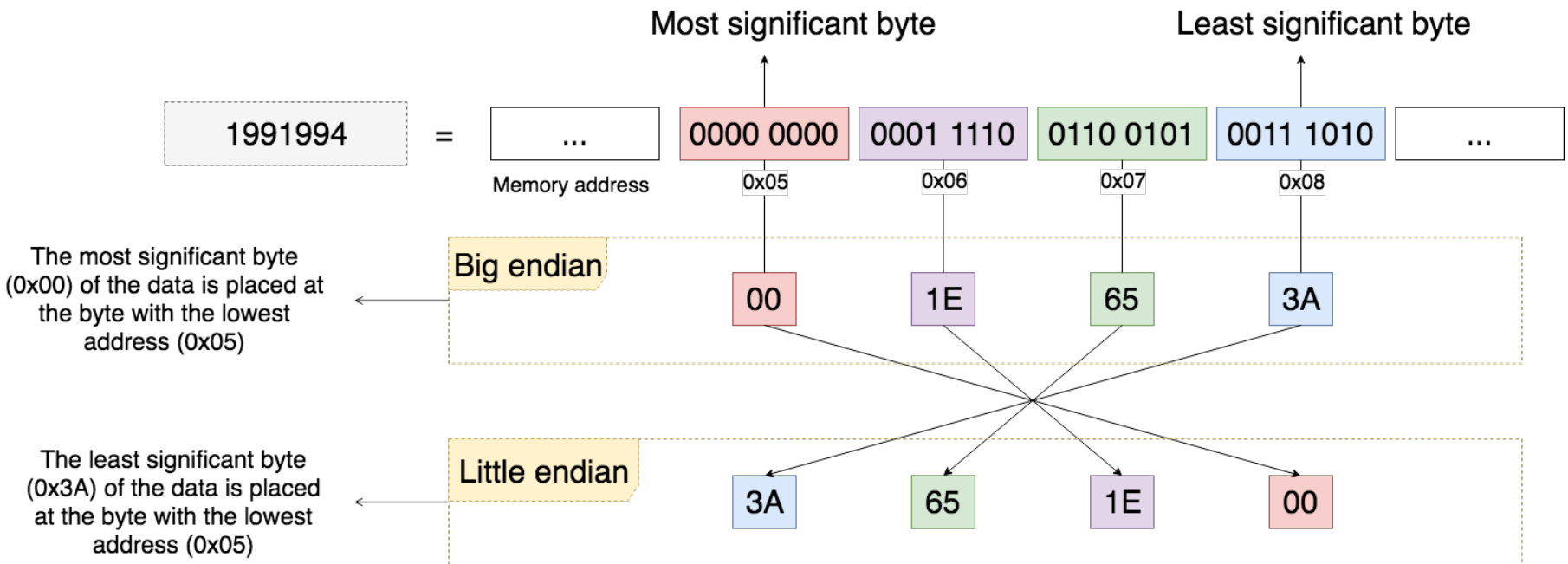
- Typed languages, such as C, subdivide the universe of data values into sets of distinct type.
- A data type defines:
  - how the values are stored and
  - how the operations on those values are performed.
- C has four primitive data types:
  - int (one word): On a 32-bit machine, an int occupies 4 bytes.
  - char (1 byte)
  - float (4 bytes)
  - double (8 bytes)

Qualifiers

- We can qualify the int data type so that it contains a minimum number of bits.
- Qualifiers:
  - short :at least 16 bits
  - long: at least 32 bits
  - long long: at least 64 bits
  - long double: at least 64 bits
- Standard C does not specify that a long double must occupy a minimum number of bits, only that it occupies no less bits than a double.

Representation of Integral Values

- C stores integral values in equivalent binary form.
- Non-Negative values:
  - Intel machines use this little-endian ordering.
  - Motorola machines use big-endian ordering - left to right.



Negative and Positive Values

Unsigned Ints

- We can use all of the bits available to store the value of a variable.
  - unsigned short
  - unsigned int
  - unsigned long
  - unsigned long long
- With unsigned variables, there is no need for a negative-value encoding scheme.

Cultural Symbols

- We store cultural symbols using an integral data type.
- We store a symbol by storing the integer associated with the symbol.
- Over 60 encoding sequences have already been defined. We use the ASCII encoding sequence throughout this course.

Encoding Sequence	Full Name	# Bits	Defined In
UCS-4	Universal Multiple-Octet Coded Character Set	32	1993
BMP	Basic Multilingual Plane	16	1993
Unicode	Unicode	16	1991
ASCII	American Standard Code for Information Interchange	7	1963
EBCDIC	Extended Binary Coded Decimal Interchange Code	8	1963

- We use the ASCII encoding sequence throughout this course.

ASCII Encoding Sequence

Ctrl	Dec	Hex	Char	Code
^@	0	00		NUL
^A	1	01		SOH
^B	2	02		STX
^C	3	03		ETX
^D	4	04		EOT
^E	5	05		ENQ
^F	6	06		ACK
^G	7	07		BEL
^H	8	08		BS
^I	9	09		HT
^J	10	0A		LF
^K	11	0B		VT
^L	12	0C		FF
^M	13	0D		CR
^N	14	0E		SO
^O	15	0F		SI
^P	16	10		DLE
^Q	17	11		DC1
^R	18	12		DC2
^S	19	13		DC3
^T	20	14		DC4
^U	21	15		NAK
^V	22	16		SYN
^W	23	17		ETB
^X	24	18		CAN
^Y	25	19		EM
^Z	26	1A		SUB
^[	27	1B		ESC
^\	28	1C		FS
^]	29	1D		GS
^^	30	1E	▲	RS
^-	31	1F	▼	US

Dec	Hex	Char
32	20	
33	21	!
34	22	"
35	23	#
36	24	$
37	25	%
38	26	&
39	27	'
40	28	(
41	29	)
42	2A	*
43	2B	+
44	2C	,
45	2D	-
46	2E	.
47	2F	/
48	30	0
49	31	1
50	32	2
51	33	3
52	34	4
53	35	5
54	36	6
55	37	7
56	38	8
57	39	9
58	3A	:
59	3B	;
60	3C	<
61	3D	=
62	3E	>
63	3F	?

Dec	Hex	Char
64	40	@
65	41	A
66	42	B
67	43	C
68	44	D
69	45	E
70	46	F
71	47	G
72	48	H
73	49	I
74	4A	J
75	4B	K
76	4C	L
77	4D	M
78	4E	N
79	4F	O
80	50	P
81	51	Q
82	52	R
83	53	S
84	54	T
85	55	U
86	56	V
87	57	W
88	58	X
89	59	Y
90	5A	Z
91	5B	[
92	5C	\
93	5D	]
94	5E	^
95	5F	_

Dec	Hex	Char
96	60	`
97	61	a
98	62	b
99	63	c
100	64	d
101	65	e
102	66	f
103	67	g
104	68	h
105	69	i
106	6A	j
107	6B	k
108	6C	l
109	6D	m
110	6E	n
111	6F	o
112	70	p
113	71	q
114	72	r
115	73	s
116	74	t
117	75	u
118	76	v
119	77	w
120	78	x
121	79	y
122	7A	z
123	7B	{
124	7C	|
125	7D	}
126	7E	~
127	7F	⌂

Declarations

Syntax

- Syntax: data_type identifier [= initial value];
- For example:

```
char section;
int numberOfClasses;
double cashFare = 2.25;
```

- In allocating memory for variables of identical data type, we may group the identifiers in a single declaration and separate them with commas. For example:

```
char section, letter, initial, answer;
int numberOfClasses, children, books, rooms;
double cashFare, money, height, weight;
```

Declaration

Naming Conventions

- must start with a letter or underscore (_)
- may contain any combination of letters, digits and underscore (_)
- must not be a C reserved word (key words)
- some compilers allow more than 31 characters, while others do not. To be safe, we avoid using more than 31 characters.

In-Class Practice

- Which of the following is an invalid identifier: whale, giraffe's, camel_back, 4me2, _how_do_you_do, senecac.on.ca, digt3, register
- Select a descriptive identifier for and write a complete declaration for:
  - A shelf of books
  - A cash register
  - A part_time student
  - A group of programs

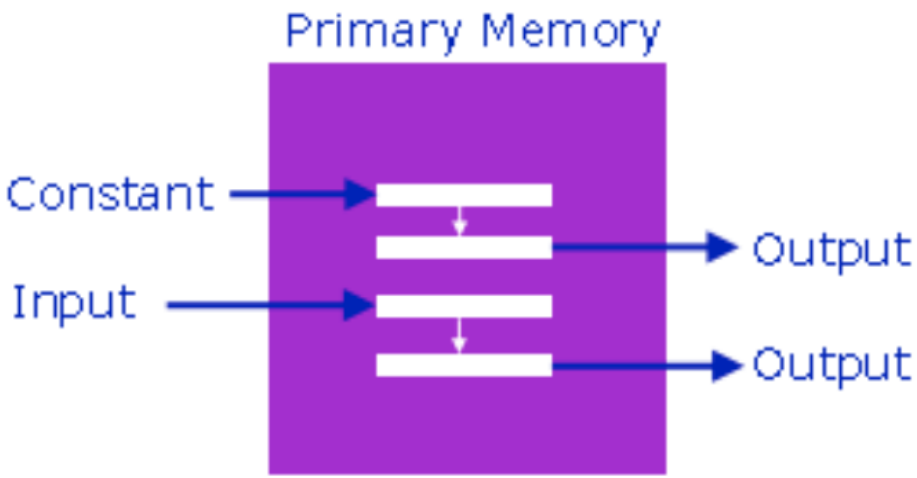
Summary

- Variables
  - Data Types
  - Integral Types
  - Floating-Point Types
  - Declarations

Q&A

Basic Memory Operations

- The C compiler allocates space for program variables in primary memory.
- We work with variables in four principal ways. For example, we can
  - assign a constant value to a variable,
  - assign the value of another variable to a variable,
  - output the value of a variable,
  - input a fresh value into a variable's memory location.



Constants

Numeric

- The compiler embeds constants directly into the program instructions and does not allocate memory for constants.
- Numeric: We identify the data type of a numeric constant by a suffix, if any, on the value itself.

Type	Size	Suffix	Example
int	1 word	none	1456234
long	32 bits	L or l (ell)	75456234L
long long	64 bits	LL or ll (ell ell)	75456234678LL
unsigned		u or U	75456234U
double	2 words	none	1.234
float	32 bits	F or f	1.234F

Constants

Character

- We define a character constant in any one of four ways:

```
// the character itself enclosed in single quotes
char c1 = 'A';
// the corresponding decimal value in the encoding sequence
char c2 = 65;
// the corresponding hexadecimal value in the encoding sequence
char c3 = 0x41;
// the corresponding octal value in the encoding sequence
char c4 = 0101;
```

Constant

Escape Sequences

- We represent special actions or characters that are difficult to write directly by escape sequences:

Character	Sequence	ASCII	EBCDIC
alarm	\a	7	47
backspace	\b	8	22
form feed	\f	12	12
newline	\n	10	37
carriage return	\r	13	13
horizontal tab	\t	9	5
vertical tab	\v	11	11
backslash	\\	92	*
single quote	\'	39	125
double quote	\"	34	127
question mark	\?	63	111

Constant

Literal String

- We represent a literal string by enclosing it in double quotes. For example, “FPT University\n”

Assignment Operators

- We use the assignment operator to store a value in a program variable. The form of assignment is

  variable = variable, constant or expression

- For example:

```
int age;
age = 18;
```

Output

- To output data to the standard output device, we use a statement of the form

  printf(format string , variable , ... , variable);

- The format string itself consists of the characters to be displayed interspersed with conversion specifiers.

Specifier	Output As A	Use With Data Type
%c	character	char
%d	decimal	char, int
%u	decimal	unsigned int
%o	octal	unsigned char, int, short, long
%x	hexadecimal	unsigned char, int, short, long
%hd	short decimal	short
%ld	long decimal	long
%lld	very long decimal	long long
%f	floating-point	float
%lf	floating-point	double
%le	exponential	double

Output

Example

```c
#include <stdio.h>


int main()
{
    int age = 18;
    double cashFare = 2.25;

    printf("His age is %d\nThe cash fare is $%lf\n", age, cashFare);

    return 0;
}
```

Output:

```
His age is 18
The cash fare is $2.250000
```

Input

- The statement to get data from the standard input device:

  scanf(format string , address , ... , address);

- The format string is a literal string. After the format string, we list the memory address of each variable into which we want the input stored. The prefix & denotes 'address of'.
- For example:

```
scanf("%d", &age);
```

Input/Ouput Example

```c
#include <stdio.h>


void main()
{
    int age;
    double cashFare;

    // Input
    printf("Enter the boy\'s age : "); // prompt for age
    scanf("%d", &age);                 // accept age
    printf("Enter the cash fare :");   // prompt for cash fare
    scanf("%lf", &cashFare);           // accept cash fare

    // Output
    printf("His age is %d\nThe cash fare is $%lf\n", age,cashFare);
}
```

In-Class Practice

- Write a program that prompts the user for the amount of money in their pockets, accepts the user input, and displays the amount in the format shown below. If the user enters 4.52, your program displays:

```
How much money do you have in your pockets?  4.52
The amount of money in your pockets is $4.52
```

Summary

- Basic Memory Operations
  - Constants
  - Assignment Operator
  - Output
  - Input

Q&A

Expressions

- A simple expression consists of an operator and operand(s).
- A compound expression consists of several operators and several operands.
- The operands may be variables, constants and/or other expressions.
- Any expression evaluates to a single value, to which we may refer on the right side of an assignment.
- For example: result = (a + b) * c
  - +, *: operators
  - a, b, c: operands

Expressions

- The compiler translates all expressions into sets of simple instructions that the Arithmetic Logic Unit (ALU) can process.
- The ALU can only evaluate the simplest expression that consist of an operator and one or two operands. If there are two operands, they must be of identical data type.
- The ALU receives the operator from the Control Unit. The operator may be
  - arithmetic,
  - relational or
  - logical.

Arithmetic Expressions

- Integral Data Types: The int and char data types accept 5 binary and 2 unary arithmetic operators
  - Binary: +, -, *, /, %
  - Unary: +, -

Arithmetic Expressions

```c
#include <stdio.h>

void main()
{
    int intRight, intLeft, intResult;
    double fptRight, fptLeft, fptResult;

    /* Input */
    printf("Enter an integer : ");
    scanf("%d", &intLeft);
    printf("Enter an integer : ");
    scanf("%d", &intRight);
    printf("Enter a floating-point number : ");
    scanf("%lf", &fptLeft);
    printf("Enter a floating-point number : ");
    scanf("%lf", &fptRight);

    /* Evaluations */
    intResult = intLeft * intRight;
    fptResult = fptLeft * fptRight;

    /* Output */
    printf("%d * %d = %d\n", intLeft, intRight, intResult);
    printf("%le * %le = %le\n", fptLeft, fptRight, fptResult);
 }
```

Statistics

- Division typically uses more resources. To avoid division, we multiply rather than divide.
- Moreover, we prefer integral operations to floating-point ones.

Relational Expressions

- Relational expressions compare values and represent conditions with a true or false result.
- The C language interprets the value zero as false and any other value as true.
- Each primitive data type admits 6 relational operators for comparing values of that data type to other values of the same data type.
- Operators: $==, >, <, >=, <=, !=$

Relational Expressions

```c
/* Relational Expressions
 *  relational.c
 *  BTP100
 *  Jan 21 2005
 */

void main( ) {
    int age, childTicket, seniorTicket;

    printf("What is your age ? ");
    scanf("%d", &age);

    childTicket = age <= 12;
    seniorTicket = age >= 65;

    printf("You need a child  Ticket (1 for yes, 0 for no) : %d\n", childTicket);
    printf("You need a senior Ticket (1 for yes, 0 for no) : %d\n", seniorTicket);
 }
```

Logical Expressions

- Logical expressions compare conditions and yield true or false results.
- We use logical operators to express compound conditions.
- The C language has 2 binary logical operators (&&, ||) and 1 unary operator (!).

Logical Expressions

```
void main( )
{
   int age, atSchool, childTicket, studentTicket, adultTicket, seniorTicket;

   printf("What is your age ? ");
   scanf("%d", &age);
   printf("Are you at school (1 for yes, 0 for no) ? ");
   scanf("%d", &atSchool);

   childTicket = age <= 12;
   studentTicket = age > 12 && age <= 19 && atSchool == 1;
   seniorTicket = age >= 65;
   adultTicket = !childTicket && !studentTicket && !seniorTicket;

   printf("You need a child   Ticket (1 for yes, 0 for no) : %d\n", childTicket);
   printf("You need a student Ticket (1 for yes, 0 for no) : %d\n", studentTicket);
   printf("You need a senior  Ticket (1 for yes, 0 for no) : %d\n", seniorTicket);
   printf("You need an adult  Ticket (1 for yes, 0 for no) : %d\n", adultTicket);
}
```

Shorthand Assignment Operators

- The C language includes a set of shorthand assignment operators, which combine arithmetic expressions with assignments.

Operator	Shorthand	Longhand	Meaning
+=	age += 4	age = age + 4	add 4 to age
-=	age -= 4	age = age - 4	subtract 4 from age
*=	age *= 4	age = age * 4	multiply age by 4
/=	age /= 4	age = age / 4	divide age by 4
%=	age %= 4	age = age % 4	remainder after age/4
++	age++ or ++age	age = age + 1	increment age by 1
--	age-- or --age	age = age - 1	decrement age by 1

Shorthand Assignment Operators

- The pre-fix operator changes the value of the operand before the value is used, while the post-fix operator changes the value of the operand after the value has been used.

```
int a = 1;
int b = a++;
int c = ++a;

printf("%d %d %d", a, b, c); // 3 1 3
```

Mixed Data Types

- Although the ALU does not perform operations on operands of differing data type directly, C compilers can interpret expressions that contain operands of differing data type.
- If a binary expression contains operands of differing type, a C compiler changes the data type of one of the operands to match the other.
- Data type hierarchy:

double	higher
float	...
long	...
int	...
char	lower

Mixed Data Types

Assignment Expressions

- If the data type of the variable on the left side of an assignment operator differs from the data type of the right side operand, the compiler
  - promotes the right operand to the data type of the left operand if the left operand is of a higher data type than the right operand,
  - truncates the right operand to the data type of the left operand if the left operand is of a lower data type than the right operand.
- For example:

```
float a = 1.6;
int b = a;
float c = b;

printf("%.1f %d %.1f", a, b, c); // 1.6 1 1.0
```

Mixed Data Types

Arithmetic and Relational Expressions

- If the operands in an arithmetic or relational expression differ in data type, the compiler promotes the value of lower data type to a value of higher data type before implementing the operation.

	(Right Operand)				
(Left Operand)	double	float	long	int	char
double	double	double	double	double	double
float	double	float	float	float	float
long	double	float	long	long	long
int	double	float	long	int	int
char	double	float	long	int	char

Table 1: Data Type of Promoted Operand

- For example:
  - 1034 * 10L yields 10340L (a long result)
  - 1034 * 10.f yields 10340.0f (a float result)

Casting

- Casting is the way to change the data type of any constant or variable.
- Syntax: (desired data type) constant or variable.
- For example:

```
int a = 4, b = 3;
double c, d;

c = a / b;
d = (double) a / b;

printf("%d %d %.2lf %.2lf", a, b, c, d); // 4 3 1.00 1.33
```

Precedence

- The rules of precedence define the order in which a compiler must decompose a compound expression.
- The compiler evaluates the first operation with the operator that has highest precedence.

Operator	Evaluate from
++ -- (post)	left to right
++ -- (pre) + - & ! (all unary)	right to left
(data type)	right to left
* / %	left to right
+ -	left to right
< ≤ > ≥	left to right
== !=	left to right
&&	left to right
||	left to right
= += -= *= /= %=	right to left

Precedence

- Use ( ) to instruct the compiler to evaluate the expression within the parentheses first. For example:
  - $2 + 3 * 5 \rightarrow 2 + 15 = 17$
  - $(2 + 3) * 5 \rightarrow 5 * 5 = 25$

Summary

- Expressions
  - Arithmetic
  - In-Class Problem
  - Statistics
  - Relational
  - Logical
  - Shorthand Assignment Operators
  - Mixing Data Types
  - Casting
  - Precedence

Q&A